Repowire: A Persistent Mesh for Heterogeneous Coding Agents

Prassanna Ganesh Ravishankar [1] [co-author line]

Abstract

Coding-agent sessions run in separate repositories and on different runtimes. Coordinating them requires an address for each session and a way to deliver messages into it. We present Repowire, a local daemon that connects Claude Code, Codex, OpenCode and Pi sessions through a common protocol. The daemon routes by peer ids distinct from display names and runtime session ids. Runtime bridges deliver messages through native input APIs. Only requests require explicit closure; notifications do not. Liveness repair runs on incoming requests rather than a periodic timer. Human control surfaces and orchestrator sessions use the same protocol as other peers. We study one operator's deployment over 146 calendar days, containing 2,810 requests and 370 agent names across 86 projects. One name appears over 144 days and 49 registrations; the median lifetime in the other-agent registration class is 12.6 minutes. Recorded request closure is 87.4% in the Python era and 98.1% in the Go era. Of agent-to-agent requests, 52% cross runtimes. Among requests with a preceding request from the same sender, 74.7% repeat its recipient. Human surfaces originate 53.5% of non-machine requests and no notifications. A separate written record contains 76 incidents, of which 34 concern identity, liveness or delivery. Repowire is open source.

1. Introduction

A developer working across repositories may run a separate coding-agent session in each repository. When a task crosses a repository boundary, the developer relays questions and answers between sessions (Ravishankar, 2026a). These relays require the developer to preserve details such as flags and version numbers. Shared documentation and task summaries provide context in advance, but must be updated when the underlying code changes.

An agent can instead ask the session working on the relevant repository. The system must resolve the recipient's address even after reconnects or restarts and deliver the question through that runtime's input interface. It must also record whether the recipient answered.

Repowire runs as a daemon on the developer's machine. Agent sessions register as *peers*. The daemon routes messages to their native input interfaces and records the exchanges in a journal. A Telegram bot, a Slack bot and a web dashboard register with the `human` role. An *orchestrator* is a peer that coordinates work through the same message protocol.

The design changed in response to operational failures. Early versions routed by display name and terminal pane. The operator's notes record 76 incidents, including stale registrations and messages delivered to the wrong session. We describe the mechanisms introduced to address these failures and use the journal to measure the resulting deployment.

Contributions.

1. A design for persistent, addressable coding agents across runtimes (Section 3): daemon-minted identity with separate registration and addressing checks and explicit retirement; a protocol in which only asks have a lifecycle; bridges under a transport-neutral router; lazy repair instead of liveness polling; humans and orchestrators as peers.

2. An open-source implementation (Section 4) that grew from 5 to 33 tools and from one runtime to six over ten months, with journal schemas that differ across the native cutover.

3. A measurement study of the mesh from its journal (Section 5): 146 days, 2,810 requests, 2,472 agent registrations. We report lifetime and identity churn, closure and latency, the sender-recipient structure of traffic including the human's share, cross-runtime use, the review loop, and incidents coded against the MAST taxonomy (Cemri et al., 2025).

4. Design lessons (Section 6) stated as claims with the evidence that supports or limits them.

[1]Independent Researcher. Correspondence to: Prassanna Ganesh Ravishankar <me@prassanna.io>.

Preprint. Draft 2. 16 October 2026. For review.

The study covers one mesh operated by the author (Section 8). The journal records sender and recipient identities, request closure and registration activity. Prior studies examine coding agents through repository activity (Agarwal et al., 2026; Mazloomzadeh et al., 2026), practitioner surveys (Pan et al., 2026), or collaborative tasks (Khatua et al., 2026). Our measurements concern a deployment used across successive tasks and sessions.

2. Setting and Goals

The operator's situation. One developer maintains several repositories (a product, its infrastructure, a few libraries, a website) and runs a coding agent in each. The agents are on different runtimes because the runtimes have different strengths and separate usage quotas. Work arrives from a phone, a laptop, or a scheduled job. Sessions are long-lived in intent and short-lived in practice: terminals close, laptops sleep, runtimes are upgraded, daemons restart.

Goals.

G1 Address. A session has a stable address that survives reconnects and daemon restarts, and that no other session can take by sharing a path or a name.

G2 Deliver. A message to a live session reaches its native input, and the sender learns whether it did.

G3 Heterogeneity. Runtimes differ in hooks, session models and input paths; the protocol must not.

G4 Humans as peers. The person is addressable and addressed by the same primitives as agents, from a phone.

G5 No polling. Agents do not send liveness heartbeats.

G6 Record. Every request, reply, registration and repair is journaled so that the mesh can be debugged and, later, measured.

G5 follows an incident in which timer-driven reconciliation marked live peers offline. Request-triggered repair can leave stale entries until another request arrives (Section 3.5).

3. Design

Figure 1 shows the components. The daemon is the only routing hub. Everything else, including human surfaces and the hosted relay, is a client. The identity rules below describe commit `880690f8` (16 October 2026), after the journal window used in the evaluation.

3.1. Peers, circles, roles

A *peer* is one running agent session: a display name derived from its working directory and runtime (`torale-codex`, suffixed on collision to `torale-2-codex`), a working path, a runtime (*backend*), a status (online, busy, offline), a per-turn state, a free-text description the agent sets, and a last-seen time. A *circle* is a routing scope that by default maps to a terminal multiplexer session, so agents started together can address each other and agents started elsewhere cannot. A *role* is metadata: `agent` by default; `human` for control surfaces; `orchestrator` for a coordinating session; `service` for service clients such as the relay. Roles that bypass circles may name a circle explicitly; an ordinary agent cannot.

3.2. Identity

The daemon routes by a minted `peer_id`. Addressing can resolve a display name to this id; registration also uses names and paths when reclaiming an offline peer or persisted mapping. Names collide across circles. Sibling terminals and subprocesses share paths. Runtime session ids can be reused on resume and are absent on some runtimes. The incident record includes failures involving these alternatives (Section 5.6).

Registration and addressing. Registration and message addressing use different checks (Table 1). Registration first checks retirement, then looks for an existing live-process holder. For Claude Code, matching process and compatible machine, backend and path can identify that holder; Codex, OpenCode and Pi also require a matching runtime session id because one process can host several sessions. The daemon next considers a claimed `peer_id` with compatible backend and path. If no id has been selected, it checks the durable binding for the backend and runtime session, subject to path compatibility. It then checks the name, circle, backend and compatible-path tuple, first among offline peers and then among persisted mappings.

Tuple reclaim adopts only a unique eligible candidate. Multiple candidates cause the daemon to mint a fresh id and journal an `identity_reclaim_ambiguous` event with the candidate ids. The fresh identity receives a distinct address when the proposed name is already held. For session-scoped runtimes, binding and tuple adoption reject candidates known to run a different session when the registrant supplies a session id. They also reject a peer whose process is alive elsewhere, including an offline record whose process still runs. Claude Code can take over a same-directory registration after the old process exits, provided the candidate remains eligible; a live sibling receives a separate, suffixed name. Addressing accepts an id or display name and rejects an ambiguous name. A daemon-minted *birth certificate*, valid for 24 hours by default, authenticates the tool server's identity claim.

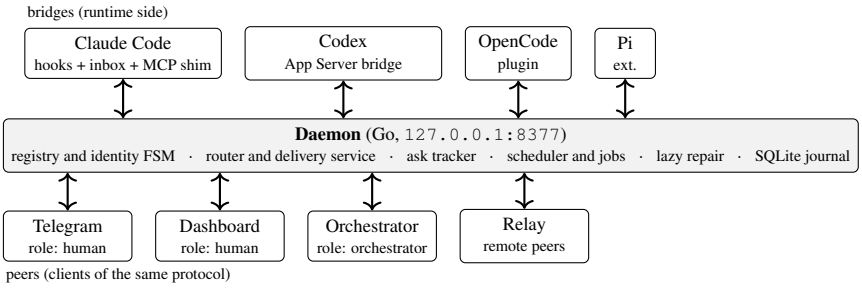


Figure 1. Components. Bridges translate each runtime's native session API into the peer protocol; the daemon resolves peer identities before transport delivery. Human surfaces, the orchestrator and the relay are clients.

Table 1. Identity checks by operation. These checks do not form a single ordered proof hierarchy.

Operation	Evidence	Effect
Addressing	peer id or unambiguous name	resolves recipient id
Tool identity	daemon-minted birth certificate	authenticates claim
Local adoption	matching pane metadata and process evidence	recovers local identity
Registration	claimed id, session binding, or unique eligible name tuple	reuses compatible identity
Kill or restart	spawn ownership or pane metadata naming the peer id	authorises pane destruction
Mapping restore	persisted mapping	restores role, circle and description

Lifecycle. Figure 2 shows the lifecycle. A clean quit, an agent-exit report, a pane takeover or three negative pane-liveness verdicts can retire an identity and sever its transport. An inconclusive verdict neither adds nor resets a strike. A soft retirement can be lifted by a live-process claim, a daemon-verified birth certificate or a compatible session binding. Tuple reclaim does not revive a retired identity. An explicit operator close creates a hard retirement that rejects reconnects claiming that id even when its process remains alive. On restart, the daemon reloads retirement records from the preceding 72 hours.

Restart. The Go daemon loads durable mappings and recent retirement records at startup, but does not recreate live peers from terminal panes. Peers register when their runtime connects. The Python implementation had a pane-hydration pass; it was removed at the Go cutover and was not ported. The Go daemon still lists orphan panes for explicit linking by the operator.

Descriptions and sessions. A peer's *description* is agent-set task state with no expiry; each turn is shown the current value and asked to update it. A *session* is the durable context

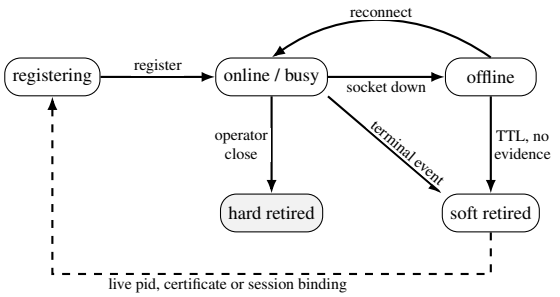


Figure 2. Go registration lifecycle. Soft retirement permits verified recovery; operator close blocks claims to the old id. Restart reloads mappings and recent retirement records, not live peers.

behind a peer: a binding from the daemon's session id to the runtime's own session id, which permits resume after the executor is gone.

3.3. Message protocol

The daemon routes four message types, chosen by lifecycle rather than content.

- `ask(to, text)` is non-blocking and returns a correlation id. The recipient closes it with `ack(id)` (seen), `ack(id, reply)` (reply delivered to the asker), or `decline(id, reason)`. Follow-ups chain with `ask(reply_to=id)`. Runtime integrations remind recipients about open asks at turn boundaries. Claude Code's Stop hook can block turn completion with a reminder when a pane id is available; OpenCode reminds on idle and Pi at a turn boundary. Prose mentions do not count as acknowledgement calls.

- `notify(to, text)` is fire-and-forget with no lifecycle.

- `broadcast(text)` normally reaches peers in the sender's circle and recipients whose roles bypass circles. A sender with a bypass role can reach other cir-

cles. Non-addressable peers are skipped, and human-inbox surfaces have separate delivery filtering.

- A *structured question* is an ask with a typed envelope (choice, text or acknowledge) answered through `answer`. Tool permission is a scope on a choice question and denies on timeout.

Peer-originated deliveries are framed so that a runtime can distinguish them from its user's instructions; messages from human surfaces are framed as user instructions. An ask to a peer the runtime says cannot accept direct input (a sub-agent thread) is refused before any state is created. Replies always reach the asker regardless of circle, because the route was fixed at ask time.

An agent explicitly closes or replies to an ask through the protocol. The daemon can also close asks on expiry, peer eviction, loss of addressability, or a job outcome. The default expiry is 24 hours. Notifications create no tracked obligation. Human surfaces sent only asks in the observed journal; agent peers used both types (Section 5.4).

3.4. Bridges and transports

A *bridge* translates between a runtime's session API and the peer protocol. Claude Code uses lifecycle hooks and an authenticated inbox for inbound delivery. A stdio shim supplies its identity when proxying tool calls to the daemon. Codex uses a supervised companion on the App Server socket to register threads and steer active turns. OpenCode and Pi load a plugin or extension inside the runtime. A *transport* is the delivery path exposed by a bridge. Failed native delivery produces a failure receipt; the bridge does not fall back to terminal keystrokes. The router resolves a peer before passing the message to its transport.

3.5. Lazy repair

Selected HTTP handlers, WebSocket connections and the MCP `list_peers` tool trigger liveness reconciliation, at most once per 30 seconds. Other MCP tools do not trigger it. Clean quits deregister through a session-end hook. A socket supervisor reports an agent process exit, and repair checks for inconsistencies missed by these paths. The registry can therefore be stale until a request triggers repair. The daemon emits a `peer_contradiction` event when it detects an inconsistency or refuses an illegal lifecycle transition. Repair deduplicates reports by peer and contradiction code. The client-side socket supervisor checks its agent process on a five-second timer; the request-triggered policy applies to daemon reconciliation, not every client activity. An operator can also request reconciliation and inspect reachability and open asks. This policy replaced timer-driven reconciliation after a spurious signal marked 43 peers offline (Section 5.6).

3.6. Humans and orchestrators as peers

Control surfaces register with the `human` role, bypass circles, and use the same four message types. A message from the phone arrives at an agent as a user instruction; an agent's reply to the phone is an `ack` with text. Structured questions render as buttons. A *human inbox* (October 2026) queues asks addressed to the human across surfaces.

An orchestrator is a peer with `role=orchestrator`; coordination is performed by the session. It is a long-running session the operator addresses from a phone, which scans a board, dispatches asks or durable jobs to project peers, tracks replies, requests reviews, and closes out. Its workspace holds communication preferences, project scope, and memory files; the daemon performs a bounded lexical recall over them and prefixes matching entries to inbound messages. A second orchestrator can coexist as an observer.

3.7. Schedules, jobs, and the journal

A *schedule* delivers a future notification or tracked ask. A *job* is a durable work record with an executor. In the Go implementation, dispatch uses the unregistered sender id `daemon-job-runner`, not a registered `@jobs` peer. A user-role chat event containing the work and attempt markers arms the job. The next assistant chat event from the assigned executor completes the armed work. Claude Code's Stop hook posts both events at turn end, so arming and completion can occur during the same Stop; the Codex bridge can post the user event as it occurs. A terminal executor failure closes active work with `executor_died`. Only dispatch is lease-bounded; delivered work is not reaped by a clock.

The journal (G6) is an append-only event table in SQLite plus a per-message delivery-trace ledger. Trace stages record creation and recipient resolution, transport outcomes such as `websocket_sent` or `thread_input_accepted`, and closure. There is no generic `delivered` stage. Resolved ids and status fields allow a misroute to be diagnosed from the record. Section 5 uses this journal as the measurement instrument.

4. Implementation

Repowire is a single Go binary (daemon, CLI, hooks, bridges, tool server, Telegram and Slack clients, relay client) with an embedded dashboard; the first eight months were Python, and the native cutover (PR #382, 10 August 2026) splits the journal into two eras with different schemas (Section 5.1). State is SQLite with versioned migrations, whose application dates on the studied host record the journal's schema changes: event journal (22 May 2026), tracked work (25 May), delivery traces (30 May), retired identities (11 June), peer provenance (18 Septem-

ber), routing-decision index (11 October), human inbox (13 October). The tool surface grew from 5 tools in December 2025 (`ask_peer`, `notify_peer`, `broadcast`, `list_peers`, `register_peer`) to 33 in October 2026, with the largest step (8 to 28) in May 2026 when asks gained lifecycle (`ack`, `answer`, `ask_many`), jobs and schedules arrived, and the orchestrator role was introduced. Six runtimes have registered peers; Gemini CLI and Antigravity were later retired as supported backends. A hosted relay tunnels dashboard and socket traffic so that a mesh on a home machine is operable from elsewhere; remote agents can join through a relay-side tool server. The repository has 828 commits and 362 pull requests from December 2025 to October 2026, including 18 pull requests from external contributors, and is Apache-licensed. [artifact URL]

5. Evaluation

We measure registration lifetimes and restart activity (Section 5.2), request closure (Section 5.3), traffic by sender and recipient (Section 5.4), and cross-runtime use (Section 5.5). We then examine the written incident record (Section 5.6).

5.1. Method and data

The data are a backup of the daemon's state database taken on 14 October 2026 and opened immutable; the daemon log, from which only start markers were extracted; the repository's history and pull-request record; and the operator's written notes (project file, 150 orchestrator memory files, 46 vault notes, two repository memory directories), from which incidents were compiled. Configuration and credential files were not read. The journal spans 22 May to 14 October 2026 (146 calendar days; 93 with any event; 64 with an active agent peer). It changes shape at the native cutover: era A (22 May to 10 August, Python) journals asks as events and replies as notification text, and has no registration events; era B (10 August to 14 October, Go) has registration and status events, typed ack events, and the delivery-trace ledger. Asks are unified across eras (2,810: 1,505 machine dispatch from the job runner, 1,305 other). We report per era where the measure depends on the schema. Sender classes: *human* (Telegram, dashboard, Slack, inbox), *orchestrator* (role or name), *peer* (other agents), *machine* (job runner), *relay* (remote tool server). Scripts and the backup copy are kept with the paper's working files; the backup is not released because ask text contains the operator's private material. Section 8 lists what the journal cannot answer.

Activity. Figure 3 shows monthly volume by sender class. Two regimes are visible: May and June, when the mesh ran many agents and a job runner (925 of June's 1,321 asks are machine dispatch; July is almost entirely machine), and August to October, when the native daemon ran a smaller mesh without jobs. These periods differ in both traffic volume and workload; the era comparison does not control for those differences.

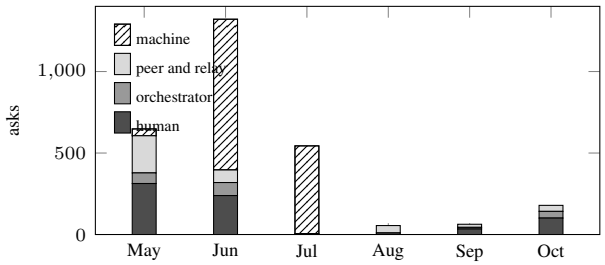


Figure 3. Asks per month by sender class, May to October 2026 (May from the 22nd, October to the 14th). Machine dispatch is the job runner, which was retired at the native cutover.

Table 2. Registration lifetime (last minus first observation of a `peer_id`), by class. Agent and orchestrator lifetimes are lower bounds on session lifetime.

class	n	p25	median	p75	p90
job executor (era A)	1,456	3.1 m	35.0 m	4.4 h	4.8 d
spawned by mesh (era B)	56	9.3 m	11.6 m	38.5 m	5.7 h
other agent	823	0	12.6 m	56.8 m	23.5 h
orchestrator	137	0	16.1 m	2.0 h	37.1 h

5.2. Registration lifetimes and name reuse

Lifetimes. A `peer_id` is a registration; one project session can hold several across reconnects, so per-id lifetimes bound session lifetime from below. Table 2 gives the distribution. The median registration in the other-agent class spans 12.6 minutes and 30% are observed exactly once; the median orchestrator registration lives 16.1 minutes. Job-executor registrations have a median observed lifetime of 35 minutes.

Names persist. Seven display names span 138 to 144 days of the 146-day calendar window. The name `repowire-claude-code` appears over 49 registrations, `orchestrator-claude-code` over 37, `repowire-codex` over 30, and `torale-codex` over 3. Senders address these names; the daemon resolves them to current registration ids. Reuse of a name does not establish continuity of a runtime session.

Restarts. Daemon restarts are not journaled, but in era B they are dated from the log: 64 starts on 14 days, 58 of them within five minutes of a burst of five or more `peer_online` events, and 41 of 60 such bursts within five minutes of a start. In era A, 243 job rows were failed by restart reconciliation at six distinct minutes: these rows record reconciliation failures, not a count of processes killed.

Table 3. Ask closure and acknowledgement latency, non-machine asks, by era.

era	asks	closed	p25	median	p75	p90
A (Python)	1,040	87.4%	14 s	26 s	60 s	2.7 m
B (Go)	265	98.1%	16 s	29 s	66 s	3.1 m

Table 4. Sender and recipient class, non-machine asks (n = 1,305).

sender	recipient	asks	share
human	orchestrator	436	33.4%
peer	peer	348	26.7%
human	peer	262	20.1%
orchestrator	peer	166	12.7%
peer	orchestrator	53	4.1%
orchestrator	orchestrator	24	1.8%
relay	orchestrator	8	0.6%
agent	human	7	0.5%
peer	machine	1	0.1%

Contradictions and repair. Lazy repair recorded 428 contradictions across the span (online without a socket 130, pane missing 126, agent process dead 5, uncoded 167) and 485 reaps, all by offline time-to-live. Offline reasons in era B are dominated by clean session end (226) and missing socket plus missing pane (115); transport failure accounts for 32. This is 2.9 recorded contradictions per calendar day across the 146-day window.

5.3. Closure and latency

Of 1,305 non-machine asks, 1,169 have a close record and 136 do not (131 in era A, 5 in era B). Recorded closure rose from 87.4% in era A to 98.1% in era B (Table 3). The design includes retirement of dead identities, explicit delivery-failure receipts and queued redelivery to a reconnecting orchestrator. Five era B asks closed with `send_failed`. The era comparison does not isolate the effect of any mechanism. Expiry is not journaled, so the open duration of the 136 unclosed asks is unknown.

Latency is time to acknowledgement, which in practice often precedes completion ("on it"); it is not task duration. The median is 26 to 29 seconds in both eras, p90 about three minutes. By sender, the human is acknowledged fastest (median 23 s, p90 105 s), peers in 38 s, the orchestrator in 40 s with a p90 of 8.3 minutes.

5.4. Who talks to whom

Table 4 gives sender and recipient classes for the 1,305 non-machine asks.

Human traffic. Human surfaces originate 53.5% of non-machine asks over the span and no notifications. Their monthly share ranges from 17% to 60%. Only seven asks run from agents to human surfaces. Agents also send ack replies and notifications, but the notification counts here classify senders, not recipients: orchestrators and peers sent 1,635 notifications in May and June.

Orchestrator traffic. Human-to-orchestrator is the largest flow, at 33.4% of non-machine asks. Peer-to-peer requests account for 26.7%. Asks sent by orchestrators rose from 11% of non-machine traffic in May to 23% in October. These shares describe the use of the protocol in this deployment.

Repeated recipients. We measure repetition over non-machine, non-self asks with a resolved recipient id. Senders are keyed by `peer_id`, except that all human surfaces form one sender. Of the 1,159 asks with a preceding ask from the same sender, 866 (74.7%) name the same recipient. Monthly shares range from 0.50 to 0.86. The repeat share is 0.859 for the human, 0.651 for peers and 0.423 for orchestrators. The companion routing study (Ravishankar, 2026b) uses a different definition: the last exchange in either direction, a 24-hour expiry and an online worker recipient. Its recency rule matches 333 of 555 recipients (0.60); that population is not a subset of the repetition measure here.

5.5. Cross-runtime traffic and review

Of 591 agent-to-agent asks, 307 (52%) cross runtimes. The recorded pairs include 177 from Claude Code to Codex, 104 in the reverse direction, 13 from Codex to Pi and 9 from Codex to Antigravity. Message-text matching identifies review traffic for 64 of the 152 repository pull requests opened during the journal span; 37 have a verdict message. Thirty have a requester and reviewer on different runtimes or verdicts from two runtimes (23 Claude Code requesters with Codex reviewers and 7 in the reverse direction). Among the 37 PRs with a verdict, 27 have one observed review round, 5 have two, 3 have three and 2 have five or more; all were merged. Some review messages postdate merge, so these counts do not establish that each review preceded it. The operator's notes estimate about 12 orchestrator turns and 30k tokens for a mediated three-round review, with about 75% saved by direct peer review. These estimates were not measured from the journal. Separately, over the repository history, the automated review bot produced 714 inline findings on 284 PRs; 80% carry a medium-severity badge.

5.6. Where failures come from

The written record contains 76 operational incidents, dated from note text where available. Three are dated before May and four are undated; the monthly inventory does not establish how many May incidents fall after the journal's 22

Table 5. Incidents from the written record by class, with the MAST mode they map to when one exists. Counts follow note density and are not a rate.

class	n	MAST mapping
peer leak or phantom identity	13	none (lifecycle)
config or binary drift	10	none (deployment)
delivery drop	8	FM-2.4 withholding, at best
misroute	7	none (lifecycle)
job or schedule failure	7	FM-3.1 premature termination, partly
daemon restart disruption	4	FM-1.4 history loss, partly
auth expiry	2	none (lifecycle)
other	25	4 credential exposures; worktree crossing; stale descriptions; review process; quota
total	76	34 lifecycle or delivery

May start. Table 5 groups incidents by proximate cause and proposes correspondences with MAST (Cemri et al., 2025). [Validate the incident-level MAST coding; the evidence inventory contains class counts, not a coding audit.]

Thirty-four incidents fall in five classes: phantom registrations (13), delivery drops (8), misroutes (7), restart disruption (4) and authentication expiry (2). These classes concern the infrastructure that keeps a recipient addressable and delivers its messages. The proposed MAST correspondences are partial: loss of history can describe a restart's effect, but does not identify the registration or delivery failure that caused it. Of the remaining 42 incidents, ten concern configuration or binary drift, seven concern jobs or schedules, and 25 are grouped as other. The latter group includes four credential exposures into transcripts or logs in October 2026. The inventory does not supply a separate count of reasoning or verification failures.

The inventory lists six matches between note incidents or incident groups and journal or log signatures. These include 43 peers marked offline on 21 June 2026 and eleven daemon starts on 13 October during a group of binary and configuration incidents. Some bursts have no matching note, and some notes have no matching burst.

5.7. Cost

The routing ledger covers roughly three days of shadow routing and records tokens rather than money. The operator's notes place the cluster hosting the relay, web and other projects at about 27 to 37 GBP per month; this is not a mesh-only cost. The inventory reports offline routing costs of 0.28 USD for Haiku, 3.67 USD for Sonnet and 7.37 USD for Opus, each over 555 decisions. Agent subscription and runtime API costs are absent from the journal.

6. Lessons

Registration identity. The 34 identity, liveness and delivery incidents motivate separating a routable registration from its display name and runtime session. In Repowire, the daemon assigns ids, checks process evidence during adoption and records retirement. Gastown also separates persistent worker identity from sessions (Yegge, 2026); Claude Code Agent Teams documents limitations on in-process teammate resumption (Anthropic, 2026). The observed closure increase does not establish which identity or delivery change caused it.

Request tracking. The ask/notify split matches the recorded traffic: humans originate asks, while agents send both asks and notifications. Explicit closure and reminders make open requests inspectable. The journal does not measure how much either mechanism contributes to the closure rate.

Request-triggered repair. Lazy repair avoids the periodic reconciliation path implicated in the mass-offline incident. It leaves registry entries stale until a request or runtime event updates them. The journal records 428 contradictions, but does not measure a bound on detection delay.

Recipient repetition. Human senders repeat their previous recipient in 86% of eligible asks, compared with 42% for orchestrators. A routing evaluation should report switches separately so that repeated recipients do not dominate its score (Ravishankar, 2026b).

Scope. The design addresses registration and delivery across successive sessions. The traffic shares and recipient repetition describe one operator's deployment. They do not estimate how other operators would use the same protocol.

7. Related Work

Task-spawned multi-agent frameworks. CAMEL, MetaGPT, ChatDev, AutoGen and OpenHands study coordination among agents within an application or task (Li et al., 2023; Hong et al., 2024; Qian et al., 2024; Wu et al., 2023; Wang et al., 2025). MAST analyses failures in multi-agent execution traces (Cemri et al., 2025). Our incident record concerns a deployment across successive sessions, including registration and delivery failures. [Check the framework comparison against supported persistence and human-interaction features.]

Coding-agent fleets. Claude Code Agent Teams uses a lead session and teammate sessions with shared tasks and messaging; its documentation lists in-process teammate

resumption as a limitation (Anthropic, 2026). Gastown separates worker identities from sessions and provides lifecycle supervision (Yegge, 2026). Cursor reports contention and coordination failures in a shared-file design before introducing separate planners and workers (Lin, 2026). Repowire routes messages across runtimes and admits human control surfaces as peers.

Protocols. Repowire exposes tools to runtimes through MCP. Interoperability surveys describe discovery and task exchange through MCP, ACP, A2A and ANP (Ehtesham et al., 2025; Sarkar & Sarkar, 2025). Our design concerns the local registration evidence and runtime input paths required to deliver messages to coding sessions. [Compare lifecycle and delivery semantics against versioned protocol specifications.]

Measurement. Pan et al. (Pan et al., 2026) study deployed agents through interviews and a practitioner survey. Repository studies examine agent contributions and their evolution (Agarwal et al., 2026; Mazloomzadeh et al., 2026; Popescu et al., 2026). CooperBench evaluates collaboration between coding agents (Khatua et al., 2026). Our journal records communication and registration activity in one deployment; it does not measure task quality. The companion paper discusses routing and delegation benchmarks (Gao et al., 2026; Ren et al., 2026; Hu et al., 2024).

Persistent state. InfiAgent reconstructs context from file-backed state (Yu et al., 2026), while Omri et al. characterise agent memory systems (Omri et al., 2026). Repowire stores orchestrator memory and runtime session bindings outside the current prompt. The learned-recall experiment is reported separately.

8. Limitations

The author operates the only mesh studied. Traffic structure may reflect that operator's habits. Incident counts depend on note density and are not failure rates. Era A has no registration events or delivery traces; 855 of the 1,169 close records depend on parsed notification text. The log cannot date most earlier daemon restarts. Ask expiry is not journaled, so 136 asks have no measured close time. Acknowledgement does not establish completion. Shared commit credentials prevent attribution of code to agents or the human. Reviews conducted only in a terminal or on GitHub are absent from the mesh review count. Fifty-three calendar days have no journal event. Runtime costs are not recorded. The private backup cannot be released; the scripts and aggregate tables are intended for the artifact. [artifact release]

Identity continuity still depends on registration order. A PID-only registration can mint a new peer before the session-start hook supplies the runtime session id. The later registration can then select that live-process holder before consulting the durable session binding, so resume does not always recover the earlier peer id (issue #474). The ambiguity fix in commit `880690f8` postdates the journal window; the reported closure rates do not evaluate it.

9. Conclusion

Repowire connects coding-agent sessions through a daemon that resolves peer identities and delivers messages through runtime bridges. Humans and orchestrators use the same request protocol. The journal records communication across six runtimes and substantial reuse of names across registrations. The incident record documents failures in identity and delivery that informed the current design. Recorded request closure increased from 87.4% to 98.1% across the Python and Go eras, but the observational comparison cannot attribute that change to individual mechanisms.

Acknowledgements

[Acknowledge the model vendors whose agents are the peers; the review bots.]

References

Agarwal, S., He, H., and Vasilescu, B. AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agents on Software Development. *arXiv preprint arXiv:2601.13597*, 2026. URL https://arxiv.org/abs/2601.13597.

Anthropic. Orchestrate teams of claude code sessions. https://code.claude.com/docs/en/agent-teams, 2026. Official documentation. Accessed 15 October 2026.

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why Do Multi-Agent LLM Systems Fail? In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2503.13657.

Ehtesham, A., Singh, A., Gupta, G. K., and Kumar, S. A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). *arXiv preprint arXiv:2505.02279*, 2025. URL https://arxiv.org/abs/2505.02279.

Gao, Y., Wang, M., Yu, Y. L., Ma, Z. C., and Qu, A. DecisionBench: A benchmark for emergent delegation in long-horizon agentic workflows. *arXiv preprint arXiv:2605.19099*, 2026.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2308.00352.

Hu, Q. J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., and Upadhyay, S. K. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.

Khatua, A., Zhu, H., Tran, P., Prabhudesai, A., Sadrieh, F., Lieberwirth, J. K., Yu, X., Fu, Y., Ryan, M. J., Pei, J., and Yang, D. CooperBench: Why coding agents cannot be your teammates yet. *arXiv preprint arXiv:2601.13295*, 2026.

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., and Ghanem, B. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2303.17760.

Lin, W. Scaling long-running autonomous coding. https://cursor.com/blog/scaling-agents, 2026. Cursor engineering post, 14 January 2026.

Mazloomzadeh, I., Morovati, M. M., and Khomh, F. How Do AI Coding Agents Contribute to Software Development? an Empirical Study of Agentic Pull Requests. *arXiv preprint arXiv:2607.21832*, 2026. URL https://arxiv.org/abs/2607.21832.

Omri, Y., Gan, Z., Broveak, Z., Geens, R., He, Z., Pentland, A., Verhelst, M., Weissman, T., and Tambe, T. Agent Memory: Characterization and System Implications of Stateful Long-Horizon Workloads. *arXiv preprint arXiv:2606.06448*, 2026. URL https://arxiv.org/abs/2606.06448.

Pan, M. Z., Arabzadeh, N., Cogo, R., Zhu, Y., Xiong, A., Agrawal, L. A., Mao, H., Shen, E., Pallerla, S., Patel, L., Liu, S., Shi, T., Liu, X., Davis, J. Q., Lacavalla, E., Basile, A., Yang, S., Castro, P., Kang, D., Sen, K., Song, D., Gonzalez, J. E., Stoica, I., Zaharia, M., and Ellis, M. Measuring Agents in Production. In *International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2512.04123.

Popescu, R. M., Gros, D., Botocan, A., Pandita, R., Devanbu, P., and Izadi, M. Investigating Autonomous Agent Contributions in the Wild: Activity Patterns and Code Change over Time. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026. URL https://arxiv.org/abs/2604.00917.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., and Sun, M. ChatDev: Communicative agents for software development. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.810. URL https://aclanthology.org/2024.acl-long.810/.

Ravishankar, P. The vibe bottleneck: AI coding workflow across multi-repo. https://prassanna.io/blog/vibe-bottleneck/, 2026a. Blog post, 23 January 2026.

Ravishankar, P. G. When routing benchmarks reward repetition: Evaluating recipient selection in a persistent agent mesh. https://github.com/prassanna-ravishankar/repowire/tree/main/paper/latex, 2026b. Companion manuscript.

Ren, Z., He, J., Zhang, X., Qian, Z., Han, K., Zheng, S., Li, G., and Zhang, X. OrchBench: Evaluating multi-agent orchestration plans in isolation via deterministic simulation. *arXiv preprint arXiv:2607.25656*, 2026.

Sarkar, A. and Sarkar, S. Survey of LLM Agent Communication with MCP: A Software Design Pattern Centric Review. *arXiv preprint arXiv:2506.05364*, 2025. URL https://arxiv.org/abs/2506.05364.

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H. H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., and Neubig, G. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2407.16741.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL https://arxiv.org/abs/2308.08155.

Yegge, S. Gas town. https://github.com/gastownhall/gastown, 2026. Software repository and documentation. Accessed 15 October 2026.

Yu, C., Wang, Y., Wang, S., Yang, H., and Li, M. InfiAgent: An Infinite-Horizon Framework for General-Purpose Autonomous Agents. *arXiv preprint arXiv:2601.03204*, 2026. URL https://arxiv.org/abs/2601.03204.

A. Tool surface over time

Table 6. MCP tools exposed to agents at month end. The Python era counts tool functions; the Go era counts registrations.

month end	tools	added	removed
2025-12	5	`ask_peer`, `broadcast`, `list_peers`, `notify_peer`, `register_peer`	
2026-01	6	`set_circle`, `whoami`	`register_peer`
2026-02	7	`kill_peer`, `spawn_peer`	`set_circle`
2026-03	8	`set_description`	
2026-05	28	`ack`, `answer`, `ask`, `ask_many`, `ask_many_result`, `claim_orchestrator_role`, `job_*` (7), `mark_reviewed`, `orchestrator_status`, `review_queue`, `schedule_*` (5)	`ask_peer`
2026-06	31	`revoke_share`, `share_session`, `wait_on_ack`	
2026-08	30		`claim_orchestrator_role`
2026-09	32	`decline`, `set_name`	
2026-10	33	`restart_peer`	

B. Surfaces by first commit

Table 7. First commit introducing each surface.

surface	date	PR
MCP server with inter-agent tools	2025-12-23	
hosted relay	2025-12-25	
web dashboard	2026-01-23	#2
circles	2026-01-25	#5
peer spawn	2026-01-27	#6
Telegram bot	2026-03-23	#30
Slack bot	2026-03-26	#32
schedules	2026-05-14	#133
SQLite state	2026-05-20	#235
orchestrator role	2026-05-21	#264
jobs	2026-05-25	#310
delivery trace ledger	2026-05-30	#318
fan-out asks	2026-05-31	#334
structured questions	2026-05-31	#335
session sharing	2026-06-11	#360
native Go daemon	2026-08-10	#382
relay-side tool server (remote agents)	2026-10-07	#410
routing decision ledger	2026-10-11	#417
human inbox	2026-10-13	#469

C. Lifecycle events by month

Table 8. Lifecycle events in the journal. Registration events exist only in era B.

month	online	offline	reaped	contradiction	spared by runtime evidence
2026-05	0	0	279	27	0
2026-06	0	318	66	122	493
2026-07	0	132	30	8	16
2026-08	450	96	29	53	100
2026-09	1,900	165	40	55	97
2026-10	1,853	179	41	163	0

D. Incident examples

One incident per class from the written record, dated by note text.

- **Peer leak or phantom** (2026-05-15): an ack from a retired orchestrator registration, absent from the peer list, arrived as a live request to merge an already-merged pull request.
- **Misroute** (2026-05-14): during co-orchestration, a project peer received three of its last four asks intended for others, and a reply addressed to a suffixed name fell back to broadcast. Fixed by the reconnect identity check.
- **Delivery drop** (2026-05-11): a reply sent on an already-closed ask returned success and was not delivered.
- **Daemon restart disruption** (2026-05-10): repeated restarts during installer testing cleared in-memory ask state; stale reminders surfaced and pane kills were refused for pre-restart peers.
- **Config drift** (2026-05-19): the installed CLI lacked a command present in the checkout; a forced reinstall left stale packages.
- **Job or schedule** (2026-06-11): a job fire never armed because the executor's turn carried no marker; the fire stayed delivered.
- **Auth expiry** (2026-06-04): a workspace token rotated to a scopeless state; after re-authentication a stale cache kept calls failing.
- **Other** (2026-05-15): one peer wrote files into another peer's worktree and switched its branch.